

# SVMCuadernoFinal

June 11, 2024

## 1 Trabajo SVM

Javier Amores Gálvez

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

import seaborn as sns; sns.set()

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report
from sklearn.utils import shuffle
from sklearn.preprocessing import LabelEncoder

from sklearn import svm, datasets
from sklearn.metrics import confusion_matrix
from sklearn import metrics
from sklearn import decomposition as de

import pandas as pd
import warnings
```

```
[2]: pip install ucimlrepo
```

```
Requirement already satisfied: ucimlrepo in /usr/local/lib/python3.10/dist-
packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.10/dist-
packages (from ucimlrepo) (2.0.3)
Requirement already satisfied: certifi>=2020.12.5 in
/usr/local/lib/python3.10/dist-packages (from ucimlrepo) (2024.6.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.10/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-
packages (from pandas>=1.0.0->ucimlrepo) (2023.4)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-
packages (from pandas>=1.0.0->ucimlrepo) (2024.1)
```

Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.0->ucimlrepo) (1.25.2)  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.16.0)

```
[3]: from ucimlrepo import fetch_ucirepo

# fetch dataset
breast_cancer_wisconsin_original = fetch_ucirepo(id=15)

# data (as pandas dataframes)
X = breast_cancer_wisconsin_original.data.features
Y = breast_cancer_wisconsin_original.data.targets

# metadata
print(breast_cancer_wisconsin_original.metadata)

# variable information
print(breast_cancer_wisconsin_original.variables)
```

```
{'uci_id': 15, 'name': 'Breast Cancer Wisconsin (Original)', 'repository_url':
'https://archive.ics.uci.edu/dataset/15/breast+cancer+wisconsin+original',
'data_url': 'https://archive.ics.uci.edu/static/public/15/data.csv', 'abstract':
'Original Wisconsin Breast Cancer Database', 'area': 'Health and Medicine',
'tasks': ['Classification'], 'characteristics': ['Multivariate'],
'num_instances': 699, 'num_features': 9, 'feature_types': ['Integer'],
'demographics': [], 'target_col': ['Class'], 'index_col':
['Sample_code_number'], 'has_missing_values': 'yes', 'missing_values_symbol':
'NaN', 'year_of_dataset_creation': 1990, 'last_updated': 'Sun Mar 10 2024',
'dataset_doi': '10.24432/C5HP4Z', 'creators': ['William Wolberg'],
'intro_paper': None, 'additional_info': {'summary': "Samples arrive periodically
as Dr. Wolberg reports his clinical cases. The database therefore reflects this
chronological grouping of the data. This grouping information appears
immediately below, having been removed from the data itself:\r\n\r\nGroup 1: 367
instances (January 1989)\r\nGroup 2: 70 instances (October 1989)\r\nGroup 3:
31 instances (February 1990)\r\nGroup 4: 17 instances (April 1990)\r\nGroup 5:
48 instances (August 1990)\r\nGroup 6: 49 instances (Updated January
1991)\r\nGroup 7: 31 instances (June 1991)\r\nGroup 8: 86 instances (November
1991)\r\n-----\r\nTotal: 699 points (as of
the donated database on 15 July 1992)\r\n\r\nNote that the results summarized
above in Past Usage refer to a dataset of size 369, while Group 1 has only 367
instances. This is because it originally contained 369 instances; 2 were
removed. The following statements summarizes changes to the original Group 1's
set of data:\r\n\r\n##### Group 1 : 367 points: 200B 167M (January
1989)\r\n\r\n##### Revised Jan 10, 1991: Replaced zero bare nuclei in 1080185 &
1187805\r\n\r\n##### Revised Nov 22,1991: Removed 765878,4,5,9,7,10,10,10,3,8,1
no record\r\n##### : Removed 484201,2,7,8,8,4,3,10,3,4,1 zero
epithelial\r\n##### : Changed 0 to 1 in field 6 of sample
```

1219406\r\n##### : Changed 0 to 1 in field 8 of following  
sample:\r\n##### : 1182404,2,3,1,1,1,2,0,1,1,1", 'purpose':  
None, 'funded\_by': None, 'instances\_represent': None, 'recommended\_data\_splits':  
None, 'sensitive\_data': None, 'preprocessing\_description': None,  
'variable\_info': '1. Sample code number: id number\r\n2. Clump  
Thickness: 1 - 10\r\n3. Uniformity of Cell Size: 1 -  
10\r\n4. Uniformity of Cell Shape: 1 - 10\r\n5. Marginal Adhesion:  
1 - 10\r\n6. Single Epithelial Cell Size: 1 - 10\r\n7. Bare Nuclei:  
1 - 10\r\n8. Bland Chromatin: 1 - 10\r\n9. Normal Nucleoli:  
1 - 10\r\n10. Mitoses: 1 - 10\r\n11. Class:  
(2 for benign, 4 for malignant)', 'citation': 'This breast cancer databases was  
obtained from the University of Wisconsin Hospitals, Madison from Dr. William H.  
Wolberg. If you publish results when using this database, then please include  
this information in your acknowledgements. Also, please cite one or more  
of:\n1. O. L. Mangasarian and W. H. Wolberg: "Cancer diagnosis via linear  
programming", SIAM News, Volume 23, Number 5, September 1990, pp 1 & 18.\n2.  
William H. Wolberg and O.L. Mangasarian: "Multisurface method of pattern  
separation for medical diagnosis applied to breast cytology", Proceedings of the  
National Academy of Sciences, U.S.A., Volume 87, December 1990, pp  
9193-9196.\n3. O. L. Mangasarian, R. Setiono, and W.H. Wolberg: "Pattern  
recognition via linear programming: Theory and application to medical  
diagnosis", in: "Large-scale numerical optimization", Thomas F. Coleman and  
Yuying Li, editors, SIAM Publications, Philadelphia 1990, pp 22-30.\n4. K. P.  
Bennett & O. L. Mangasarian: "Robust linear programming discrimination of two  
linearly inseparable sets", Optimization Methods and Software 1, 1992, 23-34  
(Gordon & Breach Science Publishers).'}}}

|    | name                        | role    | type        | demographic \ |
|----|-----------------------------|---------|-------------|---------------|
| 0  | Sample_code_number          | ID      | Categorical | None          |
| 1  | Clump_thickness             | Feature | Integer     | None          |
| 2  | Uniformity_of_cell_size     | Feature | Integer     | None          |
| 3  | Uniformity_of_cell_shape    | Feature | Integer     | None          |
| 4  | Marginal_adhesion           | Feature | Integer     | None          |
| 5  | Single_epithelial_cell_size | Feature | Integer     | None          |
| 6  | Bare_nuclei                 | Feature | Integer     | None          |
| 7  | Bland_chromatin             | Feature | Integer     | None          |
| 8  | Normal_nucleoli             | Feature | Integer     | None          |
| 9  | Mitoses                     | Feature | Integer     | None          |
| 10 | Class                       | Target  | Binary      | None          |

|   | description | units | missing_values |
|---|-------------|-------|----------------|
| 0 | None        | None  | no             |
| 1 | None        | None  | no             |
| 2 | None        | None  | no             |
| 3 | None        | None  | no             |
| 4 | None        | None  | no             |
| 5 | None        | None  | no             |
| 6 | None        | None  | yes            |
| 7 | None        | None  | no             |

|    |                           |      |      |    |
|----|---------------------------|------|------|----|
| 8  |                           | None | None | no |
| 9  |                           | None | None | no |
| 10 | 2 = benign, 4 = malignant | None |      | no |

La base de datos elegida consiste en 699 casos de posibles cánceres de mama, recogidos entre enero de 1989 y noviembre de 1991. El repositorio puede encontrarse en <https://archive.ics.uci.edu/dataset/15/breast+cancer+wisconsin+original>.

Construir reglas de clasificación para una base de datos que determine si un tumor es benigno o maligno es crucial para mejorar el diagnóstico temprano y preciso, reducir errores humanos, y optimizar recursos médicos. Esto no solo apoya a los médicos en la toma de decisiones consistentes y fundamentadas, sino que también tranquiliza a los pacientes al proporcionar información clara sobre su condición. Además, estas reglas ayudan a evitar tratamientos innecesarios, cumplen con los estándares médicos, y facilitan la mejora continua en las técnicas de diagnóstico y tratamiento, contribuyendo así a una atención de salud más eficiente y eficaz.

La variable respuesta (llamada **Class** en la base de datos) es de tipo binaria, y consiste en determinar si un tumor es benigno (denotado con un 2) o maligno (denotado con un 4).

Contamos además con 9 variables explicativas de tipo numérico, cuyos valores son enteros. Se describen a continuación:

1. **Clump\_thickness**. Mide el grosor de los grupos de células.
2. **Uniformity\_of\_cell\_size**. Evalúa la uniformidad del tamaño de las células.
3. **Uniformity\_of\_cell\_shape**. Evalúa la uniformidad de la forma de las células.
4. **Marginal\_adhesion**. Mide la adhesión marginal de las células.
5. **Single\_epithelial\_cell\_size**. Mide el tamaño de las células epiteliales.
6. **Bare\_nuclei**. Sus valores son enteros que representan la cantidad de núcleos sin citoplasma circundante. Destacar que esta variable contiene datos faltantes.
7. **Bland\_chromatin**. Evalúa la cromatina (material genético) en la célula.
8. **Normal\_nucleoli**. Mide la cantidad de nucléolos normales.
9. **Mitoses**. Sus valores son enteros que representan la frecuencia de mitosis.

[4]: X

```
[4]:      Clump_thickness  Uniformity_of_cell_size  Uniformity_of_cell_shape  \
0                5                1                1
1                5                4                4
2                3                1                1
3                6                8                8
4                4                1                1
..              ...                ...                ...
694              3                1                1
695              2                1                1
696              5               10               10
697              4                8                6
698              4                8                8

      Marginal_adhesion  Single_epithelial_cell_size  Bare_nuclei  \
0                1                2                1.0
```

```

1          5          7          10.0
2          1          2          2.0
3          1          3          4.0
4          3          2          1.0
..          ...          ...          ...
694         1          3          2.0
695         1          2          1.0
696         3          7          3.0
697         4          3          4.0
698         5          4          5.0

```

```

      Bland_chromatin  Normal_nucleoli  Mitoses
0          3          1          1
1          3          2          1
2          3          1          1
3          3          7          1
4          3          1          1
..          ...          ...          ...
694         1          1          1
695         1          1          1
696         8         10          2
697        10          6          1
698        10          4          1

```

[699 rows x 9 columns]

```
[5]: Y
```

```

[5]:      Class
0         2
1         2
2         2
3         2
4         2
..        ...
694        2
695        2
696        4
697        4
698        4

```

[699 rows x 1 columns]

A continuación se muestra un resumen de las medidas estadísticas de cada variable explicativa

```
[6]: X.describe()
```

```
[6]:
```

|       | Clump_thickness | Uniformity_of_cell_size | Uniformity_of_cell_shape | \ |
|-------|-----------------|-------------------------|--------------------------|---|
| count | 699.000000      | 699.000000              | 699.000000               |   |
| mean  | 4.417740        | 3.134478                | 3.207439                 |   |
| std   | 2.815741        | 3.051459                | 2.971913                 |   |
| min   | 1.000000        | 1.000000                | 1.000000                 |   |
| 25%   | 2.000000        | 1.000000                | 1.000000                 |   |
| 50%   | 4.000000        | 1.000000                | 1.000000                 |   |
| 75%   | 6.000000        | 5.000000                | 5.000000                 |   |
| max   | 10.000000       | 10.000000               | 10.000000                |   |

|       | Marginal_adhesion | Single_epithelial_cell_size | Bare_nuclei | \ |
|-------|-------------------|-----------------------------|-------------|---|
| count | 699.000000        | 699.000000                  | 683.000000  |   |
| mean  | 2.806867          | 3.216023                    | 3.544656    |   |
| std   | 2.855379          | 2.214300                    | 3.643857    |   |
| min   | 1.000000          | 1.000000                    | 1.000000    |   |
| 25%   | 1.000000          | 2.000000                    | 1.000000    |   |
| 50%   | 1.000000          | 2.000000                    | 1.000000    |   |
| 75%   | 4.000000          | 4.000000                    | 6.000000    |   |
| max   | 10.000000         | 10.000000                   | 10.000000   |   |

|       | Bland_chromatin | Normal_nucleoli | Mitoses    |
|-------|-----------------|-----------------|------------|
| count | 699.000000      | 699.000000      | 699.000000 |
| mean  | 3.437768        | 2.866953        | 1.589413   |
| std   | 2.438364        | 3.053634        | 1.715078   |
| min   | 1.000000        | 1.000000        | 1.000000   |
| 25%   | 2.000000        | 1.000000        | 1.000000   |
| 50%   | 3.000000        | 1.000000        | 1.000000   |
| 75%   | 5.000000        | 4.000000        | 1.000000   |
| max   | 10.000000       | 10.000000       | 10.000000  |

Como todas las variables se mueven entre los mismos valores (entre 1 y 10), no parece necesario normalizar los datos.

```
[7]: n_benign = Y[Y['Class']==2].count()[0]
n_malignant = Y[Y['Class']==4].count()[0]
print('El número de tumores benignos en la base de datos es {} ({} %)'
      .format(n_benign,100*n_benign/699))
print('El número de tumores malignos en la base de datos es {} ({} %)'
      .format(n_malignant,100*n_malignant/699))
```

El número de tumores benignos en la base de datos es 458 (65.52217453505007 %)

El número de tumores malignos en la base de datos es 241 (34.47782546494993 %)

Veamos si existe correlación lineal entre alguna de las variables.

```
[8]: correlations = X.corr()
print(correlations)
plt.matshow(correlations)
```

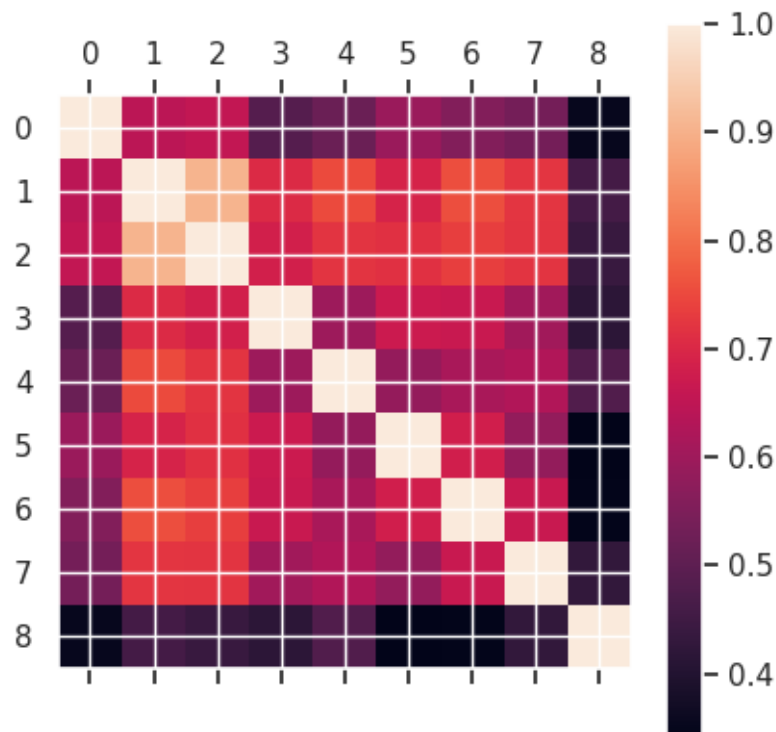
```
plt.colorbar()
plt.show()
```

|                             | Clump_thickness | Uniformity_of_cell_size \ |
|-----------------------------|-----------------|---------------------------|
| Clump_thickness             | 1.000000        | 0.644913                  |
| Uniformity_of_cell_size     | 0.644913        | 1.000000                  |
| Uniformity_of_cell_shape    | 0.654589        | 0.906882                  |
| Marginal_adhesion           | 0.486356        | 0.705582                  |
| Single_epithelial_cell_size | 0.521816        | 0.751799                  |
| Bare_nuclei                 | 0.593091        | 0.691709                  |
| Bland_chromatin             | 0.558428        | 0.755721                  |
| Normal_nucleoli             | 0.535835        | 0.722865                  |
| Mitoses                     | 0.350034        | 0.458693                  |

|                             | Uniformity_of_cell_shape | Marginal_adhesion \ |
|-----------------------------|--------------------------|---------------------|
| Clump_thickness             | 0.654589                 | 0.486356            |
| Uniformity_of_cell_size     | 0.906882                 | 0.705582            |
| Uniformity_of_cell_shape    | 1.000000                 | 0.683079            |
| Marginal_adhesion           | 0.683079                 | 1.000000            |
| Single_epithelial_cell_size | 0.719668                 | 0.599599            |
| Bare_nuclei                 | 0.713878                 | 0.670648            |
| Bland_chromatin             | 0.735948                 | 0.666715            |
| Normal_nucleoli             | 0.719446                 | 0.603352            |
| Mitoses                     | 0.438911                 | 0.417633            |

|                             | Single_epithelial_cell_size | Bare_nuclei \ |
|-----------------------------|-----------------------------|---------------|
| Clump_thickness             | 0.521816                    | 0.593091      |
| Uniformity_of_cell_size     | 0.751799                    | 0.691709      |
| Uniformity_of_cell_shape    | 0.719668                    | 0.713878      |
| Marginal_adhesion           | 0.599599                    | 0.670648      |
| Single_epithelial_cell_size | 1.000000                    | 0.585716      |
| Bare_nuclei                 | 0.585716                    | 1.000000      |
| Bland_chromatin             | 0.616102                    | 0.680615      |
| Normal_nucleoli             | 0.628881                    | 0.584280      |
| Mitoses                     | 0.479101                    | 0.339210      |

|                             | Bland_chromatin | Normal_nucleoli | Mitoses  |
|-----------------------------|-----------------|-----------------|----------|
| Clump_thickness             | 0.558428        | 0.535835        | 0.350034 |
| Uniformity_of_cell_size     | 0.755721        | 0.722865        | 0.458693 |
| Uniformity_of_cell_shape    | 0.735948        | 0.719446        | 0.438911 |
| Marginal_adhesion           | 0.666715        | 0.603352        | 0.417633 |
| Single_epithelial_cell_size | 0.616102        | 0.628881        | 0.479101 |
| Bare_nuclei                 | 0.680615        | 0.584280        | 0.339210 |
| Bland_chromatin             | 1.000000        | 0.665878        | 0.344169 |
| Normal_nucleoli             | 0.665878        | 1.000000        | 0.428336 |
| Mitoses                     | 0.344169        | 0.428336        | 1.000000 |



**Uniformity\_of\_cell\_size** y **Uniformity\_of\_cell\_shape** están fuertemente correlacionadas. En general, la matriz de correlaciones indica que ciertas características celulares, especialmente aquellas relacionadas con la uniformidad, están correlacionadas entre sí.

Antes de empezar, con el proceso de validación cruzada, tenemos que hacer algo respecto a los datos faltantes de la variable **Bare\_nuclei**. Lo más sencillo (sin meternos en técnicas avanzadas del tratamiento de falta de información), es o bien trabajar si esta variable, o bien trabajar sin las observaciones que no tengan un valor en esa variable. Lo haremos de las dos formas y posteriormente compararemos los resultados.

```
[9]: X_new = X.drop(columns=['Bare_nuclei'])
```

```
[10]: X_new
```

```
[10]:
```

|     | Clump_thickness | Uniformity_of_cell_size | Uniformity_of_cell_shape \ |
|-----|-----------------|-------------------------|----------------------------|
| 0   | 5               | 1                       | 1                          |
| 1   | 5               | 4                       | 4                          |
| 2   | 3               | 1                       | 1                          |
| 3   | 6               | 8                       | 8                          |
| 4   | 4               | 1                       | 1                          |
| ..  | ...             | ...                     | ...                        |
| 694 | 3               | 1                       | 1                          |
| 695 | 2               | 1                       | 1                          |
| 696 | 5               | 10                      | 10                         |



|     |                   |                             |                   |
|-----|-------------------|-----------------------------|-------------------|
| 697 | 4                 | 8                           | 6                 |
| 698 | 4                 | 8                           | 8                 |
|     | Marginal_adhesion | Single_epithelial_cell_size | Bland_chromatin \ |
| 0   | 1                 | 2                           | 3                 |
| 1   | 5                 | 7                           | 3                 |
| 2   | 1                 | 2                           | 3                 |
| 3   | 1                 | 3                           | 3                 |
| 4   | 3                 | 2                           | 3                 |
| ..  | ...               | ...                         | ...               |
| 694 | 1                 | 3                           | 1                 |
| 695 | 1                 | 2                           | 1                 |
| 696 | 3                 | 7                           | 8                 |
| 697 | 4                 | 3                           | 10                |
| 698 | 5                 | 4                           | 10                |
|     | Normal_nucleoli   | Mitoses                     |                   |
| 0   | 1                 | 1                           |                   |
| 1   | 2                 | 1                           |                   |
| 2   | 1                 | 1                           |                   |
| 3   | 7                 | 1                           |                   |
| 4   | 1                 | 1                           |                   |
| ..  | ...               | ...                         |                   |
| 694 | 1                 | 1                           |                   |
| 695 | 1                 | 1                           |                   |
| 696 | 10                | 2                           |                   |
| 697 | 6                 | 1                           |                   |
| 698 | 4                 | 1                           |                   |

[699 rows x 8 columns]

```
[11]: Xtrain1, Xtest1, Ytrain1, Ytest1 = train_test_split(X_new, Y, random_state = 0)
```

Realizamos la validación cruzada con 3 tipos de kernel: Lineal, polinómico y gaussiano. Los parametros que se han usado están en la lista **parametros**.

```
[12]: parametros =[ {
    'kernel': ['rbf'],
    'gamma' : [1e-4,1e-3,0.02, 0.1, 0.2, 0.5],
    'C': [1,10,100,1000]
}, {
    'kernel':["linear"],
    'C': [1,10,100,1000]
},{
    'kernel':["poly"],
    'degree': [2,3],
    'C': [1,10,100,1000]
```

```

    }]

warnings.filterwarnings('ignore')

model = GridSearchCV(svm.SVC(decision_function_shape='ovr'),
    ↪param_grid=parametros, cv=5)
model.fit(Xtrain1, Ytrain1)
model.best_params_
model.score(Xtest1, Ytest1)

```

[12]: 0.96

[13]: model.best\_params\_

[13]: {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}

El mejor modelo encontrado corresponde al del kernel gaussiano con  $\gamma = 0.001$  y  $C = 100$ .

Predecimos la variable objetivo del conjunto test con este modelo.

```

[14]: classifier = model.best_estimator_
      Y_fit1 = classifier.predict(Xtest1)

```

Vectores soporte

```

[15]: SV = classifier.support_vectors_
      print("El número de vectores soporte es {}".format(len(SV)))

```

El número de vectores soporte es 52

[16]: SV

```

[16]: array([[ 4.,  6.,  5.,  6.,  7.,  4.,  9.,  1.],
             [ 8.,  4.,  4.,  5.,  4.,  7.,  8.,  2.],
             [ 6.,  3.,  3.,  5.,  3.,  3.,  5.,  3.],
             [ 7.,  1.,  2.,  3.,  2.,  2.,  1.,  1.],
             [ 4.,  4.,  4.,  4.,  6.,  7.,  3.,  1.],
             [ 6.,  9.,  7.,  5.,  5.,  4.,  2.,  1.],
             [ 3.,  3.,  2.,  6.,  3.,  3.,  5.,  1.],
             [ 6.,  8.,  8.,  1.,  3.,  3.,  7.,  1.],
             [ 8.,  2.,  1.,  1.,  5.,  1.,  1.,  1.],
             [ 5.,  7.,  7.,  1.,  5.,  3.,  4.,  1.],
             [ 6.,  1.,  3.,  1.,  2.,  3.,  1.,  1.],
             [ 6.,  3.,  3.,  3.,  3.,  6.,  1.,  1.],
             [ 5.,  3.,  4.,  1.,  4.,  3.,  1.,  1.],
             [ 8.,  3.,  3.,  1.,  2.,  3.,  2.,  1.],
             [ 8.,  4.,  6.,  3.,  3.,  4.,  3.,  1.],
             [ 5.,  4.,  3.,  1.,  2.,  2.,  3.,  1.]])

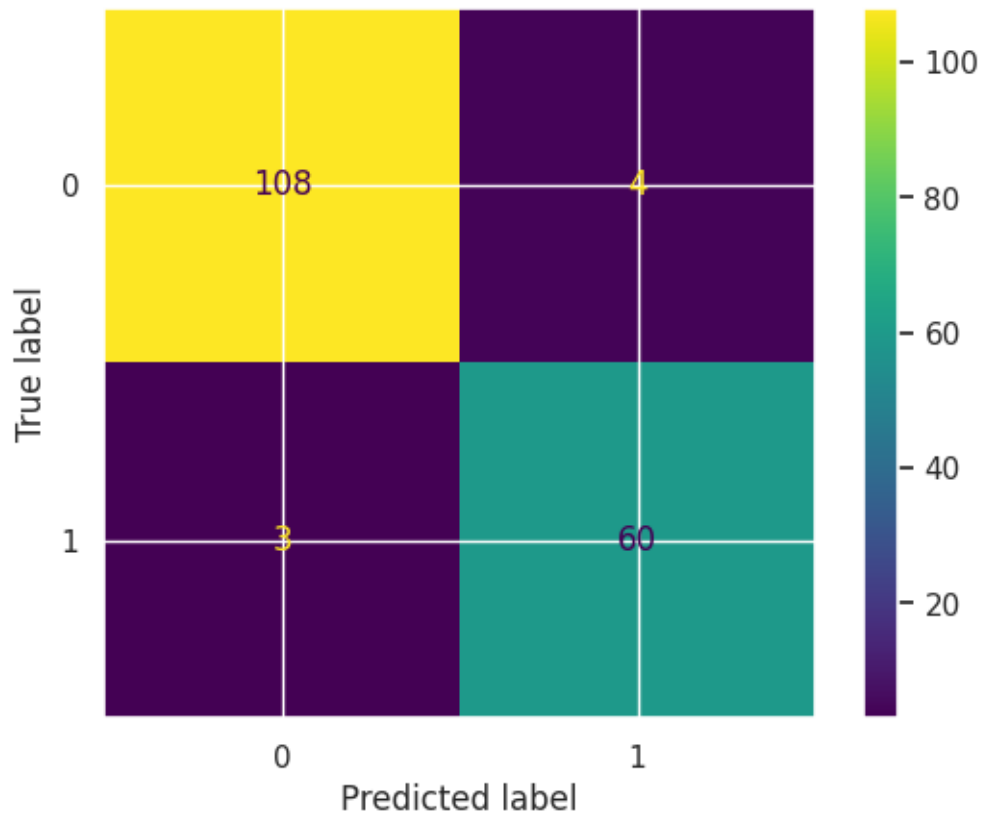
```

```
[ 5.,  3.,  4.,  3.,  4.,  4.,  7.,  1.],
[ 5.,  1.,  1.,  6.,  3.,  2.,  1.,  1.],
[ 5.,  4.,  5.,  1.,  8.,  3.,  6.,  1.],
[ 3.,  1.,  1.,  3.,  8.,  5.,  8.,  1.],
[ 4.,  3.,  1.,  1.,  2.,  4.,  8.,  1.],
[ 5.,  3.,  3.,  2.,  3.,  3.,  1.,  1.],
[ 5.,  1.,  4.,  1.,  2.,  3.,  2.,  1.],
[ 5.,  1.,  1.,  3.,  4.,  3.,  2.,  1.],
[ 1.,  3.,  3.,  2.,  2.,  7.,  2.,  1.],
[ 5.,  5.,  5.,  6.,  3.,  3.,  1.,  1.],
[ 5.,  3.,  3.,  3.,  2.,  4.,  4.,  1.],
[ 5.,  3.,  3.,  1.,  3.,  3.,  3.,  3.],
[ 5.,  3.,  3.,  3.,  6.,  3.,  1.,  1.],
[ 3.,  7.,  7.,  4.,  4.,  4.,  8.,  1.],
[ 5.,  5.,  5.,  2.,  5.,  4.,  3.,  1.],
[ 8., 10., 10.,  1.,  3.,  3.,  9.,  1.],
[ 9.,  5.,  5.,  2.,  2.,  5.,  1.,  1.],
[ 5.,  3.,  3.,  4.,  2.,  3.,  4.,  1.],
[ 1.,  4.,  3., 10.,  4.,  5.,  6.,  1.],
[10.,  2.,  2.,  1.,  2.,  1.,  1.,  2.],
[ 6.,  3.,  4.,  1.,  5.,  3.,  9.,  1.],
[ 7.,  3.,  4.,  4.,  3.,  3.,  2.,  7.],
[ 2.,  5.,  3.,  3.,  6.,  7.,  5.,  1.],
[ 3.,  4.,  5.,  2.,  6.,  4.,  1.,  1.],
[ 5.,  6.,  6.,  2.,  4.,  3.,  6.,  1.],
[ 5.,  3.,  4.,  1.,  8.,  4.,  9.,  1.],
[ 4.,  2.,  3.,  5.,  3.,  7.,  6.,  1.],
[ 8.,  6.,  4.,  3.,  5.,  3.,  1.,  1.],
[10.,  1.,  1.,  1.,  2.,  5.,  4.,  1.],
[ 5.,  2.,  3.,  4.,  2.,  3.,  6.,  1.],
[ 8., 10.,  3.,  2.,  6.,  3., 10.,  1.],
[ 3.,  3.,  6.,  4.,  5.,  4.,  4.,  1.],
[ 5.,  2.,  3.,  1.,  6.,  5.,  1.,  1.],
[ 4.,  8.,  6.,  3.,  4.,  7.,  1.,  1.],
[ 7.,  2.,  4.,  1.,  3.,  3.,  3.,  1.],
[ 6.,  1.,  3.,  1.,  4.,  5., 10.,  1.]]
```

Matriz de confusión.

```
[17]: cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix =
↪confusion_matrix(Ytest1,Y_fit1))

cm_display.plot()
plt.show()
```



```
[18]: CM = confusion_matrix(Ytest1, Y_fit1)

#TP = True positive; FP = False positive; FN = False negative; TN = True
↪negative
TP= CM[0,0]
FP= CM[1,0]
FN = CM[0,1]
TN= CM[1,1]

sen = TP/(TP+FP)
spe = TN/(TN+FN)
print("Sensitivity: ", sen)
print("Specificity: ", spe)
print("Accuracy: ", (TN+TP)/(TN+FN+TP+FP))
print("AUC", (sen+spe)/2)
```

```
Sensitivity:  0.972972972972973
Specificity:  0.9375
Accuracy:    0.96
AUC 0.9552364864864865
```

La precisión es de 0.96 y el área bajo la curva Roc es aproximadamente 0.9552, por lo que parece que el modelo es bueno.

Ahora hacemos lo mismo eliminando las observaciones con datos faltantes.

```
[19]: Aux = X.join(Y)
      XY = Aux.dropna()
      XY
```

```
[19]:      Clump_thickness  Uniformity_of_cell_size  Uniformity_of_cell_shape  \
0                    5                        1                        1
1                    5                        4                        4
2                    3                        1                        1
3                    6                        8                        8
4                    4                        1                        1
..                  ...                      ...                      ...
694                  3                        1                        1
695                  2                        1                        1
696                  5                       10                       10
697                  4                        8                        6
698                  4                        8                        8
```

```
      Marginal_adhesion  Single_epithelial_cell_size  Bare_nuclei  \
0                    1                        2            1.0
1                    5                        7           10.0
2                    1                        2            2.0
3                    1                        3            4.0
4                    3                        2            1.0
..                  ...                      ...            ...
694                  1                        3            2.0
695                  1                        2            1.0
696                  3                        7            3.0
697                  4                        3            4.0
698                  5                        4            5.0
```

```
      Bland_chromatin  Normal_nucleoli  Mitoses  Class
0                    3                  1        1      2
1                    3                  2        1      2
2                    3                  1        1      2
3                    3                  7        1      2
4                    3                  1        1      2
..                  ...                ...      ...
694                  1                  1        1      2
695                  1                  1        1      2
696                  8                 10        2      4
697                 10                  6        1      4
698                 10                  4        1      4
```

[683 rows x 10 columns]

```
[20]: X_new2 = XY.iloc[:, :9]
      Y_new2 = XY['Class']
      X_new2
```

```
[20]:      Clump_thickness  Uniformity_of_cell_size  Uniformity_of_cell_shape  \
0                    5                        1                        1
1                    5                        4                        4
2                    3                        1                        1
3                    6                        8                        8
4                    4                        1                        1
..                  ...                      ...                      ...
694                  3                        1                        1
695                  2                        1                        1
696                  5                       10                       10
697                  4                        8                        6
698                  4                        8                        8

      Marginal_adhesion  Single_epithelial_cell_size  Bare_nuclei  \
0                      1                          2           1.0
1                      5                          7          10.0
2                      1                          2           2.0
3                      1                          3           4.0
4                      3                          2           1.0
..                  ...                      ...          ...
694                  1                          3           2.0
695                  1                          2           1.0
696                  3                          7           3.0
697                  4                          3           4.0
698                  5                          4           5.0

      Bland_chromatin  Normal_nucleoli  Mitoses
0                    3                  1        1
1                    3                  2        1
2                    3                  1        1
3                    3                  7        1
4                    3                  1        1
..                  ...                ...    ...
694                  1                  1        1
695                  1                  1        1
696                  8                 10        2
697                 10                  6        1
698                 10                  4        1
```

[683 rows x 9 columns]

Ahora tan solo tenemos 683 observaciones.

```
[21]: Xtrain2, Xtest2, Ytrain2, Ytest2 = train_test_split(X_new2, Y_new2,
↳ random_state = 0)
```

```
[22]: parametros = [ {
    'kernel': ['rbf'],
    'gamma' : [1e-4, 1e-3, 0.02, 0.1, 0.2, 0.5],
    'C': [1, 10, 100, 1000]
}, {
    'kernel': ["linear"],
    'C': [1, 10, 100, 1000]
}, {
    'kernel': ["poly"],
    'degree': [2, 3],
    'C': [1, 10, 100, 1000]
}]

model2 = GridSearchCV(svm.SVC(decision_function_shape='ovr'),
↳ param_grid=parametros, cv=5)
model2.fit(Xtrain2, Ytrain2)
model2.best_params_
model2.score(Xtest2, Ytest2)
```

```
[22]: 0.935672514619883
```

Parece que también hemos obtenido un buen modelo, aunque ya podemos ver que la precisión es un poco menor que antes.

```
[23]: model2.best_params_
```

```
[23]: {'C': 1, 'gamma': 0.001, 'kernel': 'rbf'}
```

Ahora el mejor modelo encontrado corresponde al del kernel gaussiano con  $\gamma = 0.001$  y  $C = 1$ . Es decir, ahora no penalizamos tanto el error.

```
[24]: classifier2 = model2.best_estimator_
Y_fit2 = classifier2.predict(Xtest2)
```

```
[25]: SV2 = classifier2.support_vectors_
print("El número de vectores soporte es {}".format(len(SV2)))
```

El número de vectores soporte es 70

```
[26]: SV2
```

```

[26]: array([[ 5.,  1.,  3.,  3.,  2.,  2.,  2.,  3.,  1.],
 [ 1.,  1.,  1.,  1.,  2., 10.,  3.,  1.,  1.],
 [ 8.,  4.,  4.,  5.,  4.,  7.,  7.,  8.,  2.],
 [ 4.,  4.,  4.,  4.,  6.,  5.,  7.,  3.,  1.],
 [ 5.,  2.,  2.,  4.,  2.,  4.,  1.,  1.,  1.],
 [ 5.,  1.,  2., 10.,  4.,  5.,  2.,  1.,  1.],
 [ 1.,  3.,  1.,  2.,  2.,  2.,  5.,  3.,  2.],
 [ 6.,  8.,  8.,  1.,  3.,  4.,  3.,  7.,  1.],
 [ 3.,  3.,  2.,  1.,  2.,  3.,  3.,  1.,  1.],
 [ 5.,  1.,  1.,  3.,  4.,  1.,  3.,  2.,  1.],
 [ 4.,  4.,  2.,  1.,  2.,  5.,  2.,  1.,  2.],
 [ 3.,  3.,  2.,  6.,  3.,  3.,  3.,  5.,  1.],
 [ 4.,  2.,  4.,  3.,  2.,  2.,  2.,  1.,  1.],
 [ 6.,  9.,  7.,  5.,  5.,  8.,  4.,  2.,  1.],
 [ 3.,  2.,  2.,  3.,  2.,  3.,  3.,  1.,  1.],
 [ 3.,  2.,  2.,  1.,  4.,  3.,  2.,  1.,  1.],
 [ 4.,  1.,  1.,  1.,  2.,  1.,  3.,  6.,  1.],
 [ 8.,  3.,  3.,  1.,  2.,  2.,  3.,  2.,  1.],
 [ 6.,  3.,  3.,  3.,  3.,  2.,  6.,  1.,  1.],
 [ 5.,  1.,  1.,  6.,  3.,  1.,  2.,  1.,  1.],
 [ 4.,  3.,  3.,  1.,  2.,  1.,  3.,  3.,  1.],
 [ 3.,  1.,  1.,  1.,  2.,  5.,  5.,  1.,  1.],
 [ 4.,  4.,  4.,  2.,  2.,  3.,  2.,  1.,  1.],
 [ 3.,  3.,  2.,  1.,  3.,  1.,  3.,  6.,  1.],
 [ 8.,  4.,  6.,  3.,  3.,  1.,  4.,  3.,  1.],
 [ 8.,  2.,  1.,  1.,  5.,  1.,  1.,  1.,  1.],
 [ 5.,  4.,  5.,  1.,  8.,  1.,  3.,  6.,  1.],
 [ 6.,  2.,  1.,  1.,  1.,  1.,  7.,  1.,  1.],
 [ 5.,  3.,  6.,  1.,  2.,  1.,  1.,  1.,  1.],
 [ 5.,  3.,  3.,  2.,  3.,  1.,  3.,  1.,  1.],
 [ 5.,  1.,  4.,  1.,  2.,  1.,  3.,  2.,  1.],
 [ 6.,  3.,  3.,  5.,  3., 10.,  3.,  5.,  3.],
 [ 1.,  3.,  3.,  2.,  2.,  1.,  7.,  2.,  1.],
 [ 7.,  4.,  6.,  4.,  6.,  1.,  4.,  3.,  1.],
 [ 8.,  6.,  4.,  3.,  5.,  9.,  3.,  1.,  1.],
 [ 5.,  3.,  3.,  3.,  6., 10.,  3.,  1.,  1.],
 [ 4., 10.,  8.,  5.,  4.,  1., 10.,  1.,  1.],
 [ 4.,  8.,  6.,  4.,  3.,  4., 10.,  6.,  1.],
 [ 5.,  4.,  6., 10.,  2., 10.,  4.,  1.,  1.],
 [ 5.,  2.,  3.,  1.,  6., 10.,  5.,  1.,  1.],
 [ 3.,  3.,  5.,  2.,  3., 10.,  7.,  1.,  1.],
 [ 2.,  5.,  3.,  3.,  6.,  7.,  7.,  5.,  1.],
 [ 8.,  6.,  4., 10., 10.,  1.,  3.,  5.,  1.],
 [ 3.,  6.,  4., 10.,  3.,  3.,  3.,  4.,  1.],
 [ 6.,  3.,  2.,  1.,  3.,  4.,  4.,  1.,  1.],
 [10.,  4.,  2.,  1.,  3.,  2.,  4.,  3., 10.],
 [10.,  3.,  5.,  4.,  3.,  7.,  3.,  5.,  3.],

```



```

[10., 4., 3., 1., 3., 3., 6., 5., 2.],
[10., 3., 4., 5., 3., 10., 4., 1., 1.],
[ 6., 3., 4., 1., 5., 2., 3., 9., 1.],
[ 3., 3., 5., 2., 3., 10., 7., 1., 1.],
[ 5., 4., 6., 8., 4., 1., 8., 10., 1.],
[ 5., 2., 3., 4., 2., 7., 3., 6., 1.],
[ 4., 1., 1., 3., 1., 5., 2., 1., 1.],
[ 5., 3., 5., 1., 8., 10., 5., 3., 1.],
[10., 4., 5., 4., 3., 5., 7., 3., 1.],
[ 9., 5., 5., 4., 4., 5., 4., 3., 3.],
[ 5., 3., 5., 5., 3., 3., 4., 10., 1.],
[ 5., 6., 5., 6., 10., 1., 3., 1., 1.],
[ 3., 3., 6., 4., 5., 8., 4., 4., 1.],
[ 9., 5., 5., 2., 2., 2., 5., 1., 1.],
[ 5., 7., 4., 1., 6., 1., 7., 10., 3.],
[ 4., 2., 3., 5., 3., 8., 7., 6., 1.],
[ 1., 4., 3., 10., 4., 10., 5., 6., 1.],
[10., 6., 6., 3., 4., 5., 3., 6., 1.],
[ 5., 5., 5., 6., 3., 10., 3., 1., 1.],
[ 7., 2., 4., 1., 3., 4., 3., 3., 1.],
[ 7., 2., 4., 1., 6., 10., 5., 4., 3.],
[ 2., 3., 4., 4., 2., 5., 2., 5., 1.],
[10., 6., 4., 1., 3., 4., 3., 2., 3.])

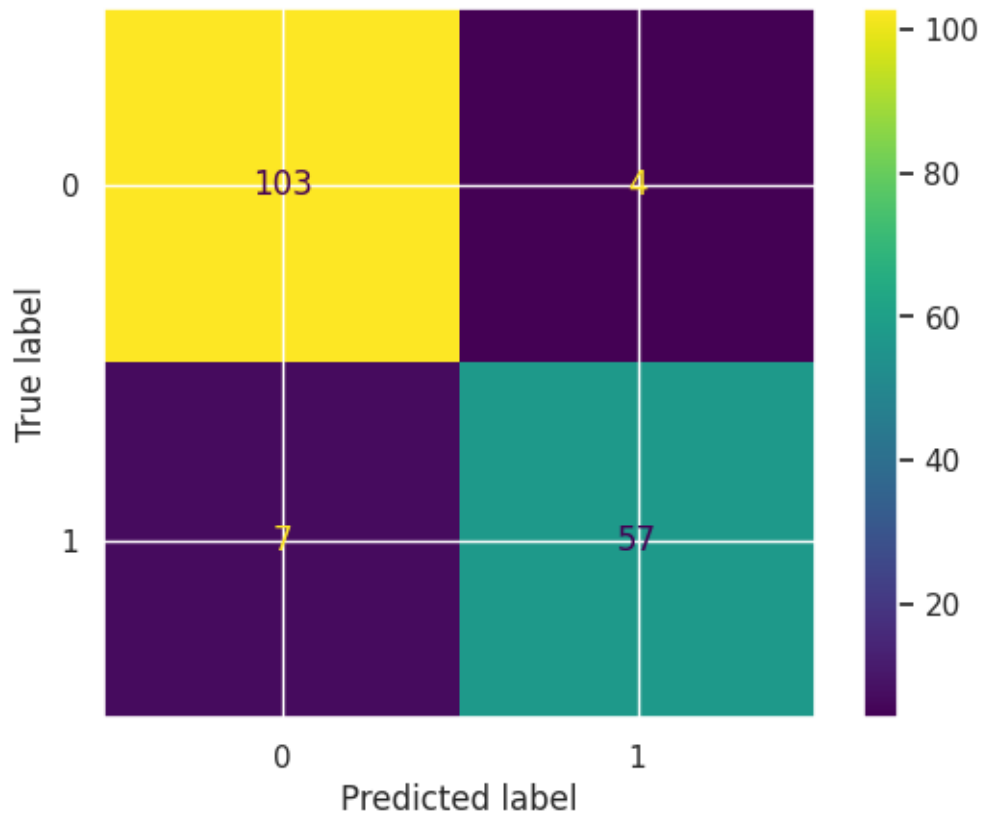
```

```

[27]: cm_display2 = metrics.ConfusionMatrixDisplay(confusion_matrix =
↳ confusion_matrix(Ytest2,Y_fit2))

cm_display2.plot()
plt.show()

```



```
[28]: CM2 = confusion_matrix(Ytest2, Y_fit2)

#TP = True positive; FP = False positive; FN = False negative; TN = True
↪negative
TP2= CM2[0,0]
FP2= CM2[1,0]
FN2 = CM2[0,1]
TN2= CM2[1,1]

sen2 = TP2/(TP2+FP2)
spe2 = TN2/(TN2+FN2)
print("Sensitivity: ", sen2)
print("Specificity: ", spe2)
print("Accuracy: ", (TN2+TP2)/(TN2+FN2+TP2+FP2))
print("AUC", (sen2+spe2)/2)
```

```
Sensitivity:  0.9363636363636364
Specificity:  0.9344262295081968
Accuracy:    0.935672514619883
AUC 0.9353949329359166
```

Este modelo también es válido, aunque da resultados ligeramente peores que el anterior.

Para terminar, vamos a intentar un método que nos permita visualizar un hiperplano. Consiste en realizar previamente el análisis de componentes principales (PCA) y quedarnos con las primeras componentes principales para ajustar el modelo.

```
[29]: from sklearn.decomposition import PCA
      from sklearn.pipeline import make_pipeline
      from sklearn.preprocessing import StandardScaler
      from sklearn.preprocessing import scale
```

```
[30]: # Entrenamiento modelo PCA
      pca_pipe = make_pipeline(StandardScaler(), PCA())
      pca_pipe.fit(X_new)

      # Se extrae el modelo entrenado del pipeline
      modelo_pca = pca_pipe.named_steps['pca']
```

```
[31]: pd.DataFrame(
      data      = modelo_pca.components_,
      columns    = X_new.columns,
      index     = ['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6', 'PC7', 'PC8']
      )
```

```
[31]:      Clump_thickness  Uniformity_of_cell_size  Uniformity_of_cell_shape \
PC1          0.318596          0.405130          0.399405
PC2         -0.168231         -0.103362         -0.129947
PC3         -0.862402          0.017994         -0.035524
PC4          0.236803         -0.131212         -0.130121
PC5          0.038930          0.131551          0.083978
PC6         -0.149956          0.109332          0.088937
PC7          0.214733         -0.469657         -0.596308
PC8          0.016406          0.746866         -0.659432
```

```
      Marginal_adhesion  Single_epithelial_cell_size  Bland_chromatin \
PC1          0.348916          0.359319          0.363963
PC2         -0.039554          0.115296         -0.264294
PC3          0.422560          0.108970          0.196867
PC4          0.655871         -0.584732          0.195801
PC5          0.245748          0.518529         -0.181008
PC6         -0.449809         -0.124396          0.736834
PC7          0.068297          0.463833          0.378277
PC8         -0.024185         -0.065753         -0.042456
```

```
      Normal_nucleoli  Mitoses
PC1          0.359195  0.249999
PC2         -0.043341  0.926035
PC3          0.129836 -0.092902
```

|     |           |           |
|-----|-----------|-----------|
| PC4 | -0.275068 | 0.153940  |
| PC5 | -0.774815 | -0.108446 |
| PC6 | -0.403295 | 0.185137  |
| PC7 | 0.115516  | -0.038553 |
| PC8 | 0.015859  | -0.010413 |

```
[32]: print('Porcentaje de varianza explicada por cada componente')
      print('-----')
      print(modelo_pca.explained_variance_ratio_)
```

Porcentaje de varianza explicada por cada componente

```
-----
[0.65982301 0.09339484 0.06735342 0.05104741 0.04695641 0.03728969
 0.03283384 0.01130138]
```

```
[33]: 0.65982301 +0.09339484
```

[33]: 0.75321785

Las dos primeras componentes principales explican el 75.32% de la varianza.

```
[34]: proyecciones = pca_pipe.transform(X=X_new)
proyecciones = pd.DataFrame(
    proyecciones,
    columns = ['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6', 'PC7', 'PC8'],
    index   = X_new.index
)
```

Observaciones en términos de componentes principales.

```
[35]: proyecciones
```

```
[35]:
```

|     | PC1       | PC2       | PC3       | PC4       | PC5       | PC6       | PC7       | \ |
|-----|-----------|-----------|-----------|-----------|-----------|-----------|-----------|---|
| 0   | -1.304088 | -0.148659 | -0.574980 | 0.223761  | -0.043164 | 0.230227  | 0.392896  |   |
| 1   | 0.916752  | -0.190742 | 0.287998  | -0.528774 | 1.433383  | -0.616196 | 0.510162  |   |
| 2   | -1.530546 | -0.029081 | 0.038016  | 0.055441  | -0.070835 | 0.336816  | 0.240264  |   |
| 3   | 1.549261  | -0.785137 | -0.619362 | -1.105117 | -0.818558 | -0.211659 | -1.577728 |   |
| 4   | -1.172750 | -0.116595 | 0.027705  | 0.599323  | 0.115253  | -0.031765 | 0.364452  |   |
| ..  | ...       | ...       | ...       | ...       | ...       | ...       | ...       |   |
| 694 | -1.666902 | 0.239961  | -0.074326 | -0.369534 | 0.312078  | -0.324203 | 0.139392  |   |
| 695 | -1.942520 | 0.247644  | 0.182924  | -0.189434 | 0.063902  | -0.214690 | -0.146546 |   |
| 696 | 4.110722  | -0.744578 | 0.645608  | -1.739107 | -0.776402 | 0.656609  | -0.610007 |   |
| 697 | 2.348565  | -1.364702 | 0.984856  | 0.156423  | -0.910494 | 1.611072  | -0.208098 |   |
| 698 | 2.666790  | -1.385565 | 1.073175  | 0.214682  | -0.025639 | 1.721435  | -0.451835 |   |

|   | PC8       |
|---|-----------|
| 0 | 0.023704  |
| 1 | -0.084927 |

```

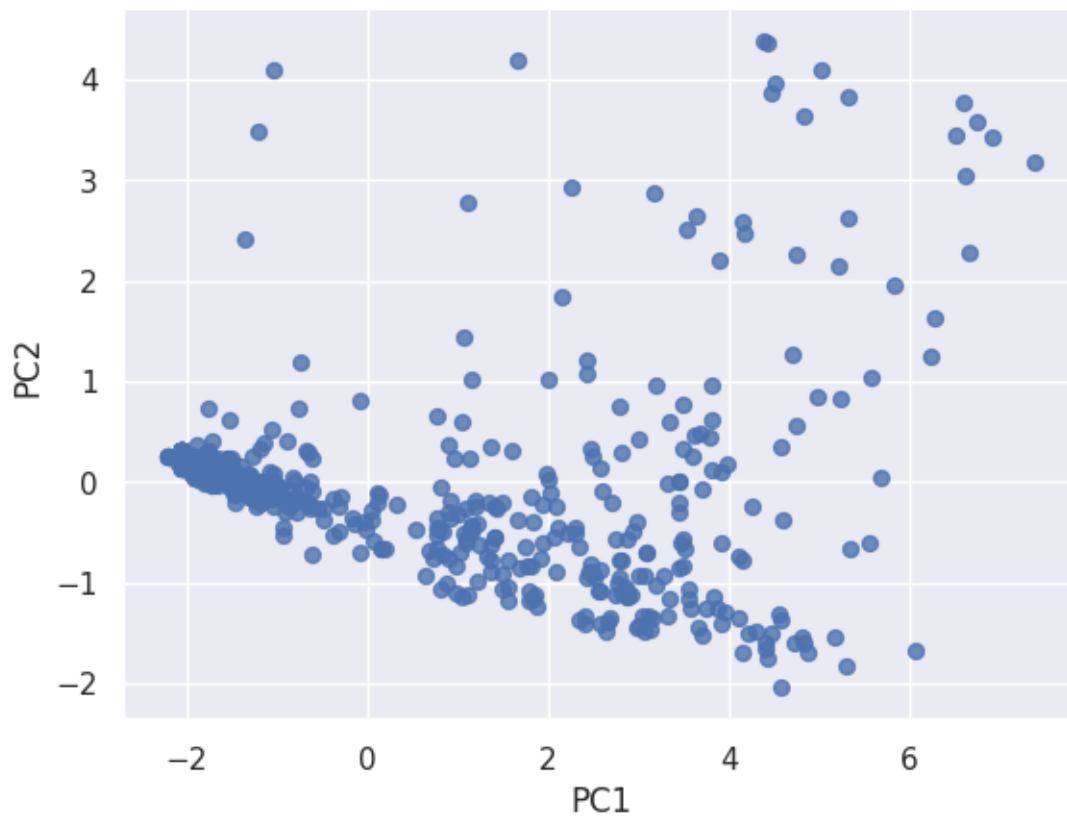
2    0.012042
3    0.191199
4    0.000921
..    ...
694  0.017175
695  0.041060
696  0.017719
697  0.471038
698 -0.021643

```

```
[699 rows x 8 columns]
```

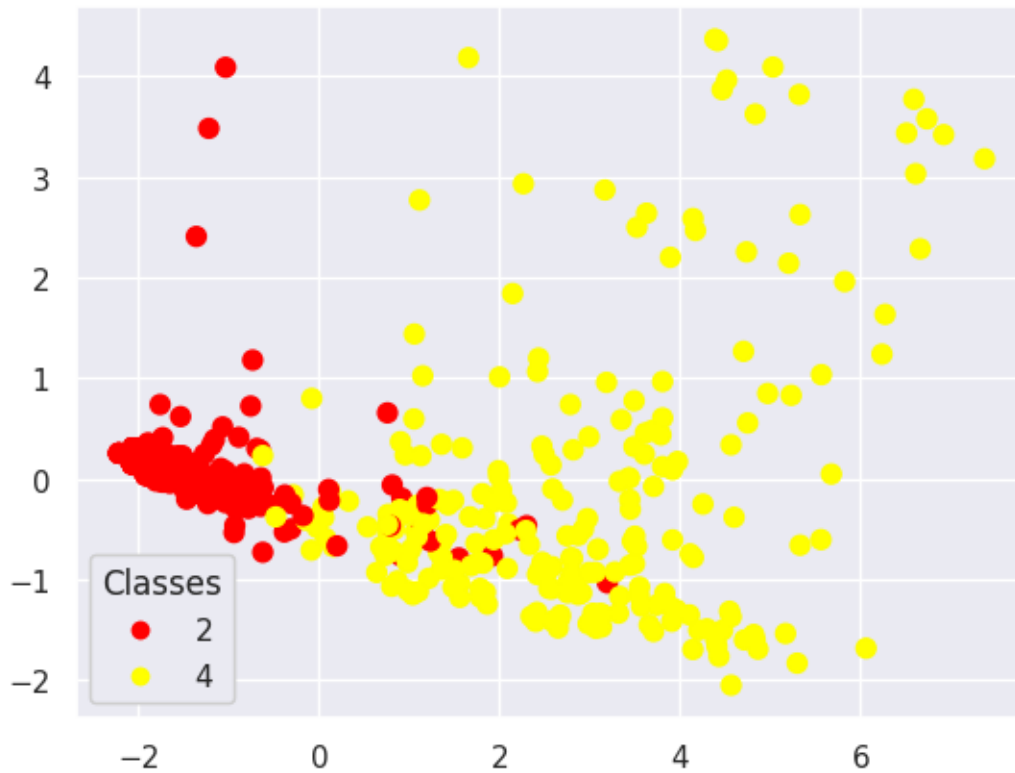
Para el SVM usaremos las dos primeras componentes principales.

```
[36]: proyecciones.plot(kind='scatter', x='PC1', y='PC2', s=32, alpha=.8)
plt.gca().spines[['top', 'right',]].set_visible(False)
```



```
[37]: color = Y["Class"]
```

```
[38]: fig, ax = plt.subplots()
scatter = ax.scatter(proyecciones.iloc[:,0], proyecciones.iloc[:,1], c = color,
                    ↪s = 50, cmap="autumn")
legend = ax.legend(*scatter.legend_elements(),
                  loc="lower left", title="Classes")
ax.add_artist(legend)
plt.show()
```



```
[39]: proyecciones.iloc[:, :2]
```

```
[39]:
```

|     | PC1       | PC2       |
|-----|-----------|-----------|
| 0   | -1.304088 | -0.148659 |
| 1   | 0.916752  | -0.190742 |
| 2   | -1.530546 | -0.029081 |
| 3   | 1.549261  | -0.785137 |
| 4   | -1.172750 | -0.116595 |
| ..  | ...       | ...       |
| 694 | -1.666902 | 0.239961  |
| 695 | -1.942520 | 0.247644  |
| 696 | 4.110722  | -0.744578 |
| 697 | 2.348565  | -1.364702 |
| 698 | 2.666790  | -1.385565 |

[699 rows x 2 columns]

```
[40]: Xtrain3, Xtest3, Ytrain3, Ytest3 = train_test_split(proyecciones.iloc[:,2], Y,
↳ random_state = 0)
```

```
[41]: parametros = [ {
    'kernel': ['rbf'],
    'gamma' : [1e-4, 1e-3, 0.02, 0.1, 0.2, 0.5],
    'C': [1, 10, 100, 1000]
  }, {
    'kernel': ["linear"],
    'C': [1, 10, 100, 1000]
  }, {
    'kernel': ["poly"],
    'degree': [2, 3],
    'C': [1, 10, 100, 1000]
  } ]

model3 = GridSearchCV(svm.SVC(decision_function_shape='ovr'),
↳ param_grid=parametros, cv=5)
model3.fit(Xtrain3, Ytrain3)
model3.best_params_
model3.score(Xtest3, Ytest3)
```

[41]: 0.96

```
[42]: model3.best_params_
```

[42]: {'C': 10, 'gamma': 0.2, 'kernel': 'rbf'}

El mejor modelo encontrado sigue siendo uno con kernel gaussiano. Sin embargo ahora los parámetros son  $\gamma = 0.2$  y  $C = 10$ .

```
[43]: classifier3 = model3.best_estimator_
Y_fit3 = classifier3.predict(Xtest3)
```

```
[44]: SV3 = classifier3.support_vectors_
print("El número de vectores soporte es {}".format(len(SV3)))
```

El número de vectores soporte es 52

```
[45]: SV3
```

```
[45]: array([[ 2.29938019, -0.46425649],
 [ 2.21891054, -0.50519419],
 [ 0.75795599,  0.65673634],
 [-0.73495438,  1.18099267],
 [ 1.23405246, -0.6172702 ],
 [ 1.92235002, -0.76396639],
 [ 0.11432234, -0.21465019],
 [ 1.54926083, -0.7851369 ],
 [ 1.14031908, -0.5008721 ],
 [-1.22063421,  3.48341034],
 [ 0.19891404, -0.66477713],
 [-0.3101204 , -0.24351056],
 [-0.31198606, -0.49753776],
 [ 0.89838368, -0.76099329],
 [-0.55047199, -0.25780263],
 [-0.37938171, -0.15954172],
 [ 0.790096   , -0.46492244],
 [ 1.19534808, -0.18375617],
 [ 0.81108547, -0.06052702],
 [-0.17823245, -0.36455498],
 [-0.4847149 , -0.26572271],
 [-0.38481881, -0.5267449 ],
 [ 0.53911963, -0.47647913],
 [-0.02230901, -0.48276849],
 [-0.07983264,  0.80038162],
 [ 6.23616892,  1.23991237],
 [ 0.12473355, -0.12326687],
 [ 1.73854623, -0.64026713],
 [ 6.05966199, -1.68171352],
 [ 0.75956071, -0.45369172],
 [ 2.02196411, -0.10930265],
 [ 0.63925882, -0.92922838],
 [-0.04939772, -0.38816327],
 [ 0.90167292,  0.37019009],
 [-0.62346531,  0.23199971],
 [ 1.9293227 , -0.22881467],
 [ 7.3724912 ,  3.18120482],
 [ 1.82858227, -1.15934154],
 [ 1.11373227,  2.77095991],
 [ 0.32720384, -0.21970316],
 [ 1.68358543, -0.86374009],
 [-0.08605852, -0.70715113],
 [ 0.05316718, -0.38267256],
 [ 2.33158061, -0.65388275],
 [ 6.58948778,  3.7722209 ],
 [ 1.80869904, -0.14563395],
 [ 0.76414964, -0.35200274],
```



```
[ 0.04605004, -0.27858007],
[ 4.41560099,  4.35339272],
[ 1.93591765, -0.60301157],
[ 2.414586   , -1.3248342 ],
[-0.1434854  , -0.40970463]])
```

Pasamos a dibujar el hiperplano

```
[46]: plt.scatter(proyecciones.iloc[:,0], proyecciones.iloc[:,1], c=color, s=50,
    cmap="autumn")

ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

##Generamos la parrilla de puntos para evaluar el modelo
xx = np.linspace(-5, 10, 300)
yy = np.linspace(-5, 10, 300)
Y_2, X_2 = np.meshgrid(yy,xx)

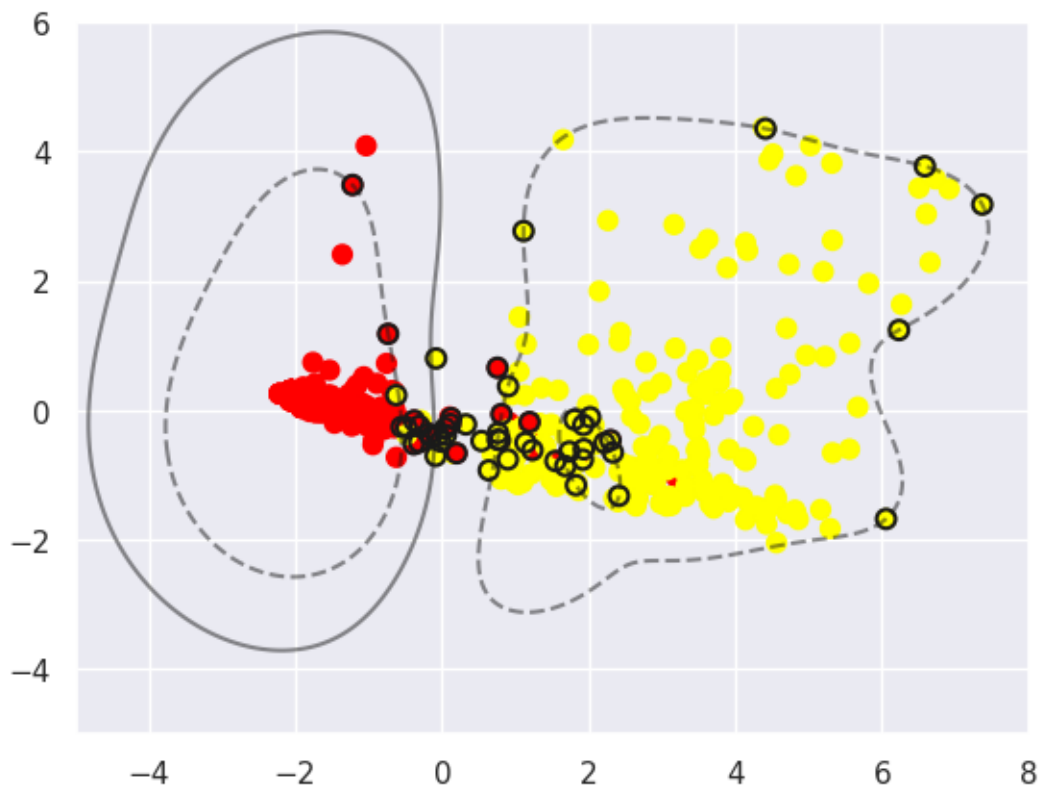
xy = np.vstack([X_2.ravel(), Y_2.ravel()]).T
P = classifier3.decision_function(xy).reshape(X_2.shape)

##Representamos las fronteras y los márgenes del SVC
ax.contour(X_2,Y_2,P, colors="k", levels=[-1,0,1], alpha = 0.5,
    linestyle=["--", "-", "--"])

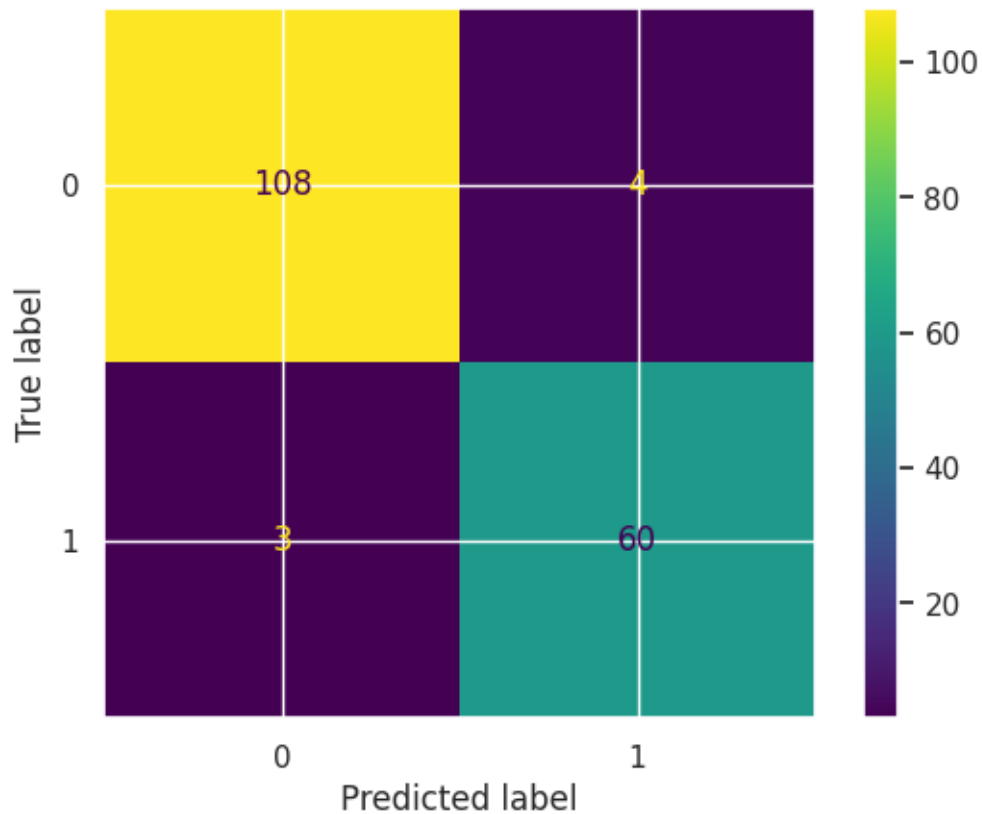
ax.scatter(SV3[:,0], SV3[:,1], s=50, linewidth=1.5, facecolors = 'none',
    edgecolor='k')

ax.set_xlim((-5, 8))
ax.set_ylim((-5,6))
```

[46]: (-5.0, 6.0)



```
[47]: cm_display3 = metrics.ConfusionMatrixDisplay(confusion_matrix =  
      ↪confusion_matrix(Ytest3,Y_fit3))  
  
cm_display3.plot()  
plt.show()
```



```
[48]: CM3 = confusion_matrix(Ytest3, Y_fit3)

TP3= CM3[0,0]
FP3= CM3[1,0]
FN3 = CM3[0,1]
TN3= CM3[1,1]

sen3 = TP3/(TP3+FP3)
spe3 = TN3/(TN3+FN3)
print("Sensitivity: ", sen3)
print("Specificity: ", spe3)
print("Accuracy: ", (TN3+TP3)/(TN3+FN3+TP3+FP3))
print("AUC", (sen3+spe3)/2)
```

```
Sensitivity:  0.972972972972973
Specificity:  0.9375
Accuracy:  0.96
AUC 0.9552364864864865
```

En cuanto a bondad de ajuste, tenemos los mismos resultados que en el primer modelo.